

**Júlia Ribeiro de Souza**

## **Desenvolvimento de Sistemas – Módulo 3 – Agenda 04**

### **Atividade – Fichário – TextChanged, Search Bar e ObservableCollection**

**Atividade feita com o auxílio da IA Google Gemini**

#### **Parte I:**

A Atividade deverá ser realizada em três etapas:

Parte 1: Utilize uma ferramenta de IA como ChatGPT, Bing AI ou Gemini para pesquisar sobre implementação de busca dinâmica em listas no .NET MAUI. Utilize para pesquisa questões como:

- Você conhece o .NET MAUI?
- Conheça sobre mecanismos de buscas?
- Como implementar o evento TextChanged em um SearchBar?
- Como filtrar dados em uma ObservableCollection em tempo real
- Como implementar um exemplo de utilização do SearchBar para permitir a busca dinâmica de produtos.

Utilize uma ObservableCollection para armazenar os produtos e garantir que a interface seja atualizada automaticamente.

#### **Respostas da IA Google Gemini:**

- **Você conhece o .NET MAUI?**  
Sim, o .NET MAUI (*Multi-platform App UI*) é o framework cross-platform da Microsoft para criar aplicativos nativos para Android, iOS, macOS e Windows a partir de um único código-fonte em C# e XAML.
- **Conhece sobre mecanismos de buscas?**  
Sim, em aplicativos mobile/desktop, os mecanismos de busca filtram coleções de dados em memória comparando o texto digitado pelo usuário com as propriedades dos objetos (como o nome ou descrição de um produto) usando LINQ.
- **Como implementar o evento TextChanged em um SearchBar?**  
No XAML, define-se o `SearchBar` associando o atributo `TextChanged` ao nome do método manipulador (*event handler*):

XML



```
<SearchBar Placeholder="Busca de Produtos" TextChanged="OnSearchBarTextChanged"
```

- Como filtrar dados em uma ObservableCollection em tempo real?  
Mantém-se uma lista principal estática/mestre com todos os produtos e uma ObservableCollection vinculada à interface ( ListView ou CollectionView ). A cada alteração do texto, a ObservableCollection é limpa ( Clear() ) e preenchida novamente apenas com os itens correspondentes do filtro LINQ:

C#



```
var filtrados = listaCompleta.Where(p => p.Descricao.ToLower().Contains(textoBusca));
produtosExibidos.Clear();
foreach (var item in filtrados)
{
    produtosExibidos.Add(item);
}
```

C#



```
listaCompleta.Where(p => p.Descricao.ToLower().Contains(textoBusca.ToLower()));
produtosExibidos.Clear();
foreach (var item in filtrados)
{
    produtosExibidos.Add(item);
}
```

- Exemplo prático de utilização do SearchBar para busca dinâmica de produtos:  
Interface (ListaProduto.xaml):

XML

```
<StackLayout Padding="10">
    <SearchBar Placeholder="Busca de Produtos" TextChanged="OnSearchBarTextChang

    <CollectionView ItemsSource="{Binding ProdutosFiltrados}">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <HorizontalStackLayout Spacing="15" Padding="5">
                    <Label Text="{Binding Id}" FontAttributes="Bold" />
                    <Label Text="{Binding Descricao}" />
                    <Label Text="{Binding Preco, StringFormat='{0:C}'}" />
                </HorizontalStackLayout>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</StackLayout>
```

Código C# (ListaProduto.xaml.cs):

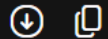
XML

```
<StackLayout Padding="10">
    <SearchBar Placeholder="Busca de Produtos" TextChanged="OnSearchBarTextChanged" />

    <CollectionView ItemsSource="{Binding ProdutosFiltrados}">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <HorizontalStackLayout Spacing="15" Padding="5">
                    <Label Text="{Binding Id}" FontAttributes="Bold" />
                    <Label Text="{Binding Descricao}" />
                    <Label Text="{Binding Preco, StringFormat='{0:C}'}" />
                </HorizontalStackLayout>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</StackLayout>
```

Código C# (ListaProduto.xaml.cs):

C#



```
using System.Collections.ObjectModel;

namespace AppMinhasCompras;

public partial class ListaProduto : ContentPage
{
    private List<Produto> listaCompleta = new();
    public ObservableCollection<Produto> ProdutosFiltrados { get; set; } = new()

    public ListaProduto()
    {
        InitializeComponent();
        CarregarDadosExemplo();
        BindingContext = this;
    }

    private void CarregarDadosExemplo()
    {
        listaCompleta = new List<Produto>
        {
            new Produto { Id = 1, Descricao = "Arroz", Preco = 25.50 },
            new Produto { Id = 2, Descricao = "Feijão", Preco = 8.90 },
            new Produto { Id = 3, Descricao = "Açúcar", Preco = 4.20 }
        };
    }
}
```

C#



```
stem.Collections.ObjectModel;

AppMinhasCompras;

partial class ListaProduto : ContentPage

{
    private List<Produto> listaCompleta = new();
    private ObservableCollection<Produto> ProdutosFiltrados { get; set; } = new();

    public ListaProduto()

    {
        InitializeComponent();
        CarregarDadosExemplo();
        BindingContext = this;
    }

    private void CarregarDadosExemplo()

    {
        listaCompleta = new List<Produto>
        {
            new Produto { Id = 1, Descricao = "Arroz", Preco = 25.50 },
            new Produto { Id = 2, Descricao = "Feijão", Preco = 8.90 },
            new Produto { Id = 3, Descricao = "Açúcar", Preco = 4.20 }
        };
    }
}
```

```
        AtualizarLista(listaCompleta);
    }

    private void OnSearchBarTextChanged(object sender, TextChangedEventArgs e)
    {
        string texto = e.NewTextValue?.ToLower() ?? "";

        var resultado = listaCompleta.Where(p => p.Descricao.ToLower().Contains(texto));
        AtualizarLista(resultado);
    }

    private void AtualizarLista(IEnumerable<Produto> produtos)
    {
        ProdutosFiltrados.Clear();
        foreach (var prod in produtos)
        {
            ProdutosFiltrados.Add(prod);
        }
    }
}
```

```

AtualizarLista(listaCompleta);

ate void OnSearchBarTextChanged(object sender, TextChangedEventArgs e)

string texto = e.NewTextValue?.ToLower() ?? "";

var resultado = listaCompleta.Where(p => p.Descricao.ToLower().Contains(texto));
AtualizarLista(resultado);

ate void AtualizarLista(IEnumerable<Produto> produtos)

ProdutosFiltrados.Clear();
foreach (var prod in produtos)
{
    ProdutosFiltrados.Add(prod);
}

```

*Utilize uma ObservableCollection para armazenar os produtos e garantir que a interface seja atualizada automaticamente.* - Esta parte está feita e se localiza no [github](#)

## **Parte II:**

- ***Quais desafios encontraram ao implementar a busca dinâmica?***

Alguns desafios foram a implantação do model com a interface, o carregamento automático dos dados e a conexão entre os eventos TextChanged e Search Bar para consultar os dados no SQLite.

- ***Como a IA ajudou no processo de aprendizado e otimização do código?***

Explicou os códigos de maneira simplificada e sua aplicação no aplicativo. Já na otimização, reduziu repetições.

- ***Quais melhorias podem ser aplicadas na funcionalidade?***

Adição de indicadores de carregamento; mensagens de que o produto não foi adicionado na lista; expandir a busca para ID ou categoria; implantar atrasos rápidos para evitar chamadas excessivas ao banco de dados.